This might seem like a lot of work for little benefit but remember that `setQuoteStash` gives us control over the incoming data. The declaration of `public array $quoteStash` means that the property can only be overwritten by other arrays, which gives a small degree of type safety. Even better, we can make use of PHP's "splat" operator `...` to enforce the data type of array elements (for a detailed explanation of "splat" see the section "Advanced Arguments" in Chapter 3, "Functions"):

```
    public function setQuoteStash(string ...$stash): void
    {
        $this->quoteStash = $stash;
    }
```

Now let's try to use some broken data with the setter:

```
$bot = new QuoteBot();
$bot->setQuoteStash(...[
    "Don't get uptight with me, man." . PHP_EOL,
    new stdClass(),
]);
```

```
Output:
Fatal error: Uncaught TypeError: QuoteBot::setQuoteStash(): Argument #2
must be of type string, stdClass given...
```

The declaration `setQuoteStash(string ...$stash)` means "take all incoming arguments and build the `$stash` array out of them, they must all be strings." What we've done is enforce an array with the class property declaration, and automatic array creation from all the data passed into `setQuoteStash()`, where we add another layer of type enforcement. As a result, our code is much safer than it was with just a public property!

This pattern of data hiding is central to OOP, and used correctly, it makes for safer and more robust code. `setQuoteStash()` provides limited safe write access to a hidden internal property, and the corollary `getQuoteStash()` (not shown here but easy enough to imagine) would provide read access. You will often hear functions like this referred to with the term "getters and setters."